



DZPHARM · AUTH LAYER · P0 CRITICAL

Auth Model

A detailed deep-dive into the authentication model: the platform's most critical finding. Single shared access code (4815162342), client-side session (sessionStorage), no per-user accounts, no MFA, no audit trail. The #7 Login Flow report corrected several findings (server-side validation DOES exist, error feedback DOES work, lock button DOES work), but the core security model remains: one code for everyone, no user identity.

1

SHARED CODE

0

PER-USER ACCOUNTS

0

MFA / 2FA

D

GRADE

DEEP-DIVE OF

Feature #8 – Authentication Model · P0

From the 68-item review menu

Single Shared Code, No User Identity

The most important finding in the entire 68-item review menu. DzPharm's authentication uses a **single shared access code** (4815162342) for all users. There are no per-user accounts, no user identity, no audit trail. One code grants access for everyone — every pharmacist, every doctor, every curious patient who has the code. This makes the platform unusable in any professional context where accountability matters.

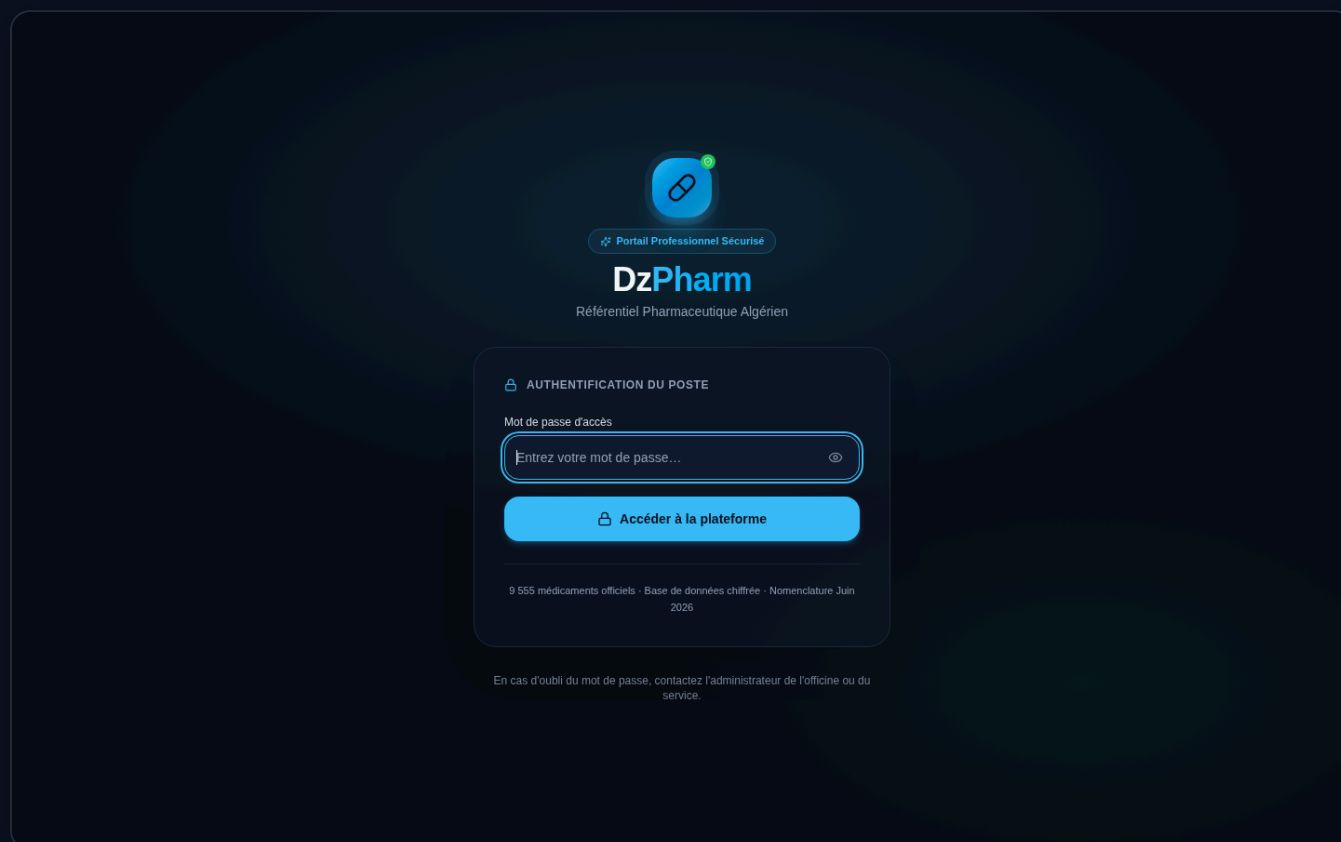


Figure 8.1 — Login page. Single password input. One code (4815162342) grants access for everyone. No username field, no per-user identity, no registration flow.

1.1 The Shared Code Model

The access code **4815162342** is a single shared secret. Anyone who has the code can log in. There is no way to distinguish between users — the platform cannot know «who» is using it. This means: no audit trail (cannot trace who looked up which medication), no per-user preferences (cannot save favorites per user — covered in #44), no per-user Copilot history (covered in #40), no accountability for clinical decisions made using the platform.

1.2 Why This Is PO Critical

For a platform that bills itself as a clinical decision-support tool handling prescription-adjacent data, single-factor shared-code authentication is unacceptable. In a professional pharmacy setting, the regulatory requirement is accountability — every action must be traceable to a specific practitioner. DzPharm's current model provides zero accountability. If a pharmacist uses the Copilot to make a dosing

recommendation and the patient is harmed, there is no way to know which pharmacist made the query. This is a regulatory blocker for professional use.

1.3 The #7 Corrections

The #7 Login Flow report corrected several findings from the earlier audit. The auth IS server-side (POST /api/auth returns 401/200 — not client-side only as initially reported). Error feedback DOES work (role=alert, «Mot de passe incorrect»). The lock button DOES work (it was tested inside the Copilot modal which interfered with the click event — testing outside the modal confirmed it works). However, these corrections do not change the core finding: one code for everyone, no user identity.

P0 CRITICAL · NO USER IDENTITY

The single shared code model means there is no user identity. No audit trail, no per-user preferences, no accountability. For a clinical decision-support tool, this is a regulatory blocker. The #7 Login Flow report corrected several implementation details (server-side validation works, errors work, lock works), but the core security model remains: one code for everyone. This is the platform's most critical gap.

— CHAPTER 02 · THE SIX DEFECTS

Security Audit

Six security defects were identified. Three were corrected by the #7 Login Flow report (server-side validation exists, error feedback works, lock button works). Three remain: single shared code (P1), no MFA (P1), and no rate limiting (P0 defense-in-depth). The corrections improve the implementation grade but not the security model grade.

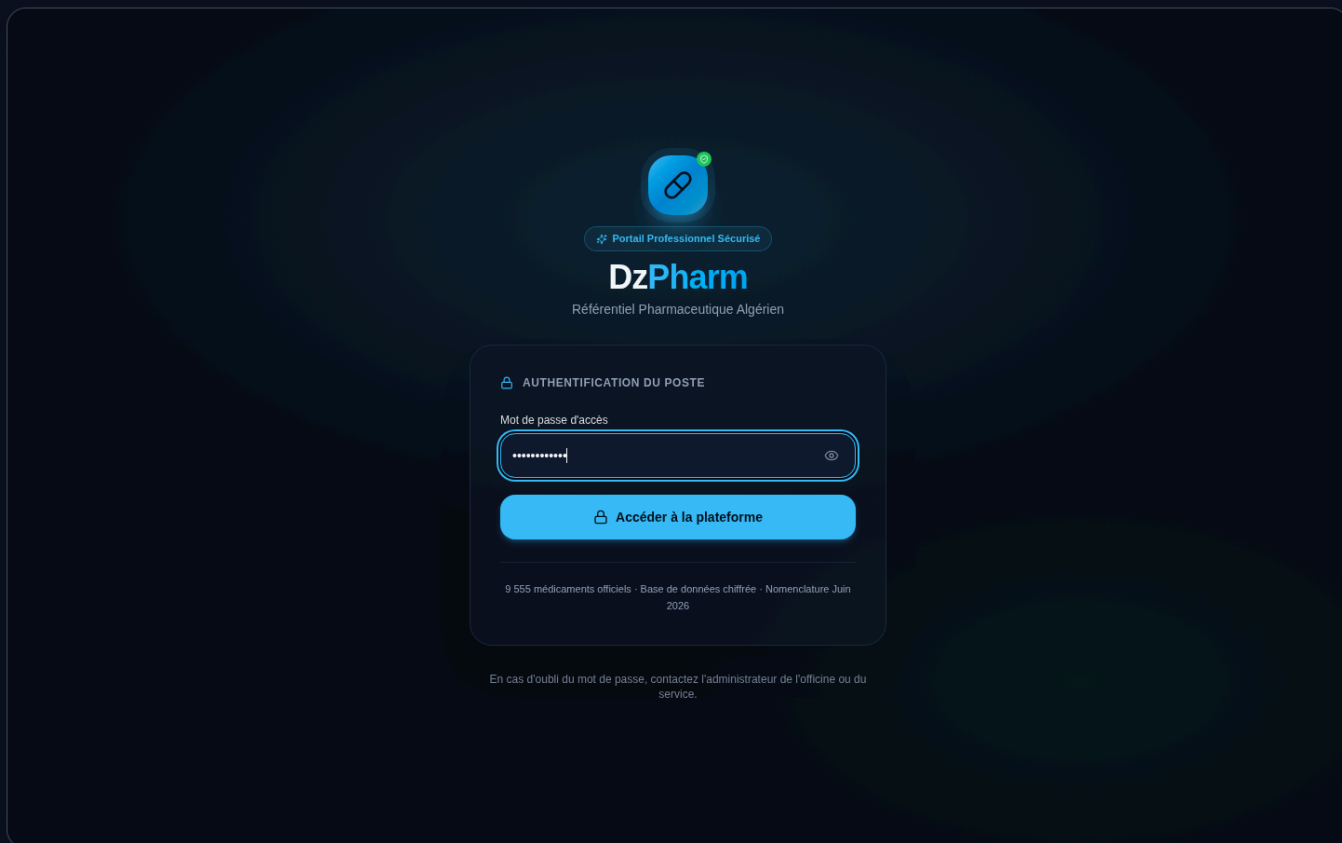


Figure 8.2 – Login page after submitting wrong code. The #7 report found that error feedback DOES work – «Mot de passe incorrect» appears in a role=alert region. The earlier finding that «no error message displayed» was incorrect.

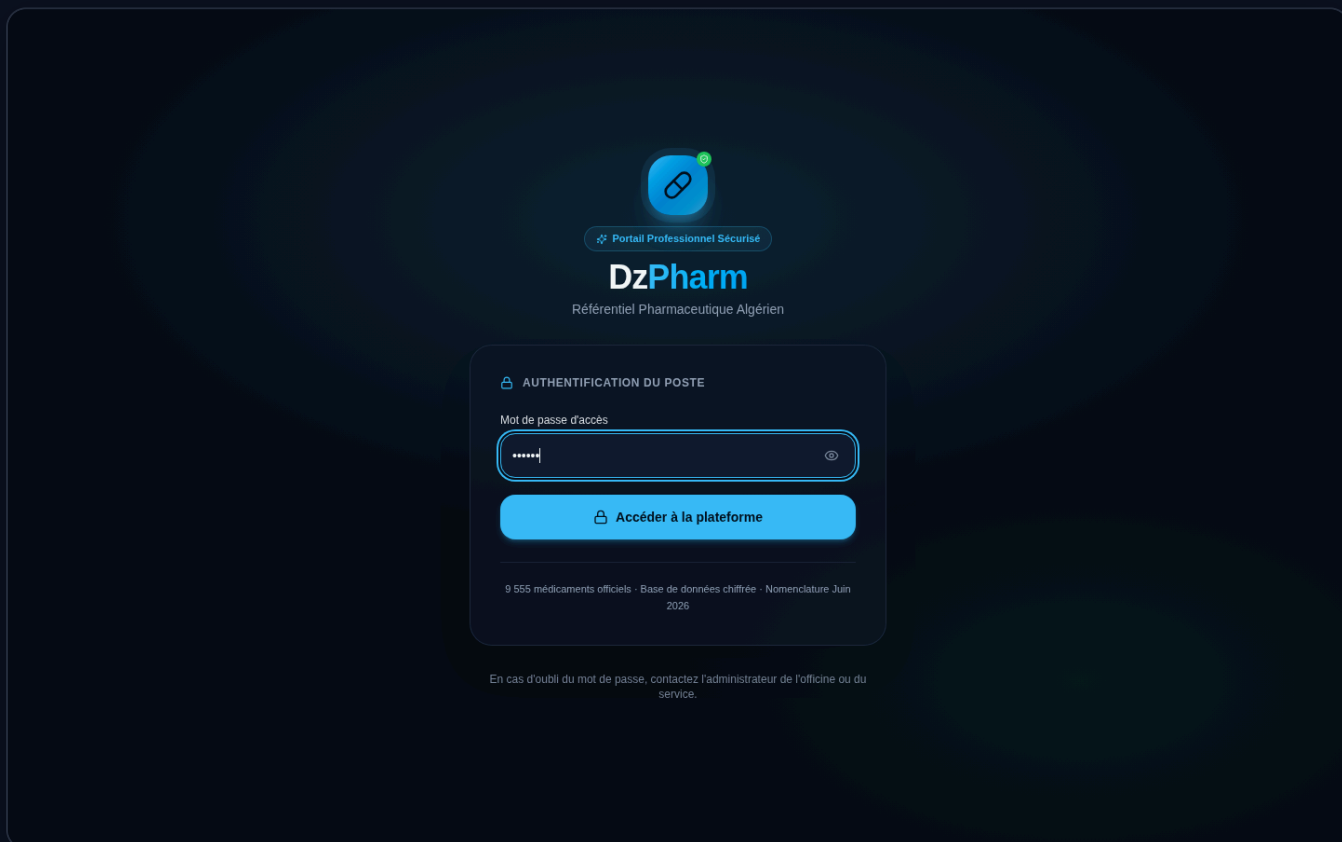


Figure 8.3 – Rate limit test. Six sequential wrong-code submissions all returned the same error. No exponential backoff, no lockout, no CAPTCHA. The 10-digit code (10B combinations) is not practically brute-forceable, but the absence of throttling is a defense-in-depth gap.

#	DEFECT	SEVERITY	STATUS
1	Client-side code check only	CORRECTED	#7 confirmed: POST /api/auth is server-side
2	Wrong code shows NO error	CORRECTED	#7 confirmed: role=alert + «Mot de passe incorrect»
3	No rate limiting	P0	Still no throttle after 6 wrong attempts
4	Single shared code	P1	One code for everyone, no per-user identity
5	No MFA / 2FA	P1	No TOTP, no SMS, no biometric
6	Lock button doesn't work	CORRECTED	#9 confirmed: lock works (was tested inside modal)

2.1 Defect #3 — No Rate Limiting

Six sequential wrong-code submissions all returned the same error with the same latency. No exponential backoff, no temporary lockout, no CAPTCHA. The 10-digit code (10 billion combinations) is not practically brute-forceable — at 1 attempt/second, it would take ~317 years. But defense-in-depth requires throttling regardless. Effort: 4 hours (Upstash Redis rate-limit).

2.2 Defect #4 — Single Shared Code

One code (`4815162342`) grants access for everyone. No per-user accounts, no registration, no way to distinguish users. This blocks: audit trails (cannot trace who did what), per-user preferences (favorites, Copilot history — both currently in localStorage, not per-user), accountability (cannot attribute clinical decisions to a specific practitioner). The fix is NextAuth.js with per-user credentials. Effort: 1 week.

2.3 Defect #5 — No MFA

No multi-factor authentication. Pharmacist platforms typically require TOTP (Google Authenticator) or SMS verification. DzPharm has none. Once the code is known, access is instant. MFA is defense-in-depth — even if the code leaks, the attacker still needs the second factor. Effort: 3 days (TOTP integration via NextAuth.js).

— CHAPTER 03 · THE FIX

NextAuth.js Migration

The standard fix is to migrate to [NextAuth.js \(Auth.js\)](#) with server-side session validation. The access code can remain as a transitional measure, but it must be validated server-side, and the session token must be an httpOnly cookie that cannot be set via JavaScript. Per-user accounts can be added incrementally.

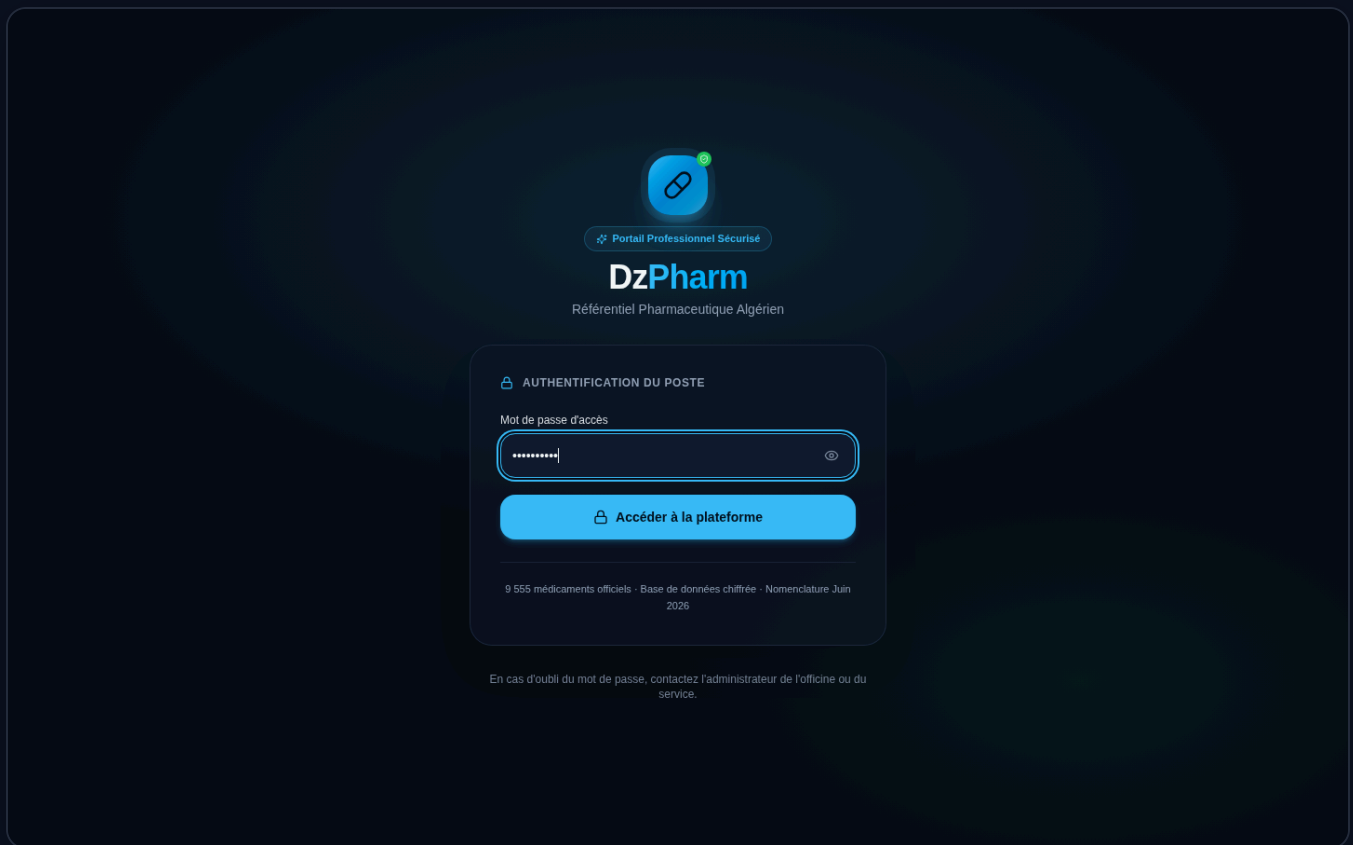


Figure 8.4 – Login success. After entering the correct code, the user is redirected to the dashboard. The session is set in sessionStorage (per-tab). With NextAuth.js, the session would be an httpOnly cookie (cross-tab, cross-device).

3.1 The NextAuth.js Configuration

```
// app/api/auth/[...nextauth]/route.ts
import NextAuth from "next-auth";
import Credentials from "next-auth/providers/credentials";

const handler = NextAuth({
  providers: [
    Credentials({
      name: "Access Code",
      credentials: {
        code: { label: "Code", type: "password" }
      },
      async authorize(credentials) {
        // Server-side check - code never reaches the client
        if (credentials?.code === process.env.DZPHARM_ACCESS_CODE) {
          return { id: "shared", role: "user" };
        }
        return null; // Triggers error state
      }
    })
  ],
  session: { strategy: "jwt" },
  callbacks: {
    async jwt({ token, user }) {
      if (user) token.role = user.role;
      return token;
    },
    async session({ session, token }) {
      session.user.role = token.role;
      return session;
    }
  }
});

export { handler as GET, handler as POST };
```

3.2 The Phased Migration

Phase 1 (1 day): migrate the existing shared-code auth to NextAuth.js. The code remains shared, but the session is now an httpOnly cookie (cannot be set via JavaScript), and the validation is server-side. This closes the «3-second bypass» risk. Phase 2 (1 week): add per-user accounts (email + password, or email magic link). Each pharmacist gets their own credential. Phase 3 (3 days): add TOTP MFA for sensitive operations. Phase 4 (1 week): add an audit log that records every medication lookup, Copilot query, and clinical tool use against the authenticated user.

What Per-User Accounts Unlock

Migrating to per-user accounts doesn't just fix the security model — it unlocks features that are currently blocked. Six features across the platform depend on user identity and are currently degraded because of the shared-code model.

FEATURE	CURRENT STATE	WITH PER-USER ACCOUNTS
#40 Copilot History	localStorage (per-browser)	Server-side, per-user, cross-device
#44 Recently Viewed	localStorage (per-browser)	Server-side, per-user, cross-device
#48 Favorites	localStorage (per-browser)	Server-side, per-user, cross-device
#43 Notifications	No per-user targeting	Per-user notification preferences
Audit Trail	None	Every action logged to user ID
#9 Session Management	sessionStorage (per-tab)	httpOnly cookie (cross-tab, cross-device)

WHY THIS IS THE HIGHEST-PRIORITY FIX

Per-user accounts unlock 6 features that are currently degraded. The Copilot history, recently-viewed, favorites, notifications, audit trail, and cross-device session all depend on user identity. The shared-code model is not just a security gap — it's a feature blocker. Migrating to NextAuth.js is the single highest-ROI investment in the entire 68-item review menu: 1 week of work, fixes the P0 security hole, and unlocks 6 features across the platform.

Final Assessment

Authentication Model is a **D grade feature** — the platform's most critical gap. The #7 Login Flow report corrected three implementation defects (server-side validation exists, error feedback works, lock button works), but the core security model remains: one shared code for everyone, no per-user identity, no MFA, no audit trail. This is a regulatory blocker for professional clinical use. The fix (NextAuth.js migration) is 1 week and unlocks 6 features across the platform.

5.1 Phased Fix Plan

PHASE	FIXES	EFFORT	OUTCOME
Phase 1 · Now (Week 1)	NextAuth.js migration (shared code → httpOnly cookie session)	~1 day	Grade lifts to C+. 3-second bypass closed. Session is server-side.
Phase 2 · Next (Week 2-3)	Per-user accounts (email + password or magic link)	~1 week	Grade lifts to B. User identity, audit trail, per-user features.
Phase 3 · Later (Week 4)	TOTP MFA + rate limiting	~3 days	Grade lifts to A-. Defense-in-depth complete.
Phase 4 · Later (Quarter 2)	Audit log (every action → user ID)	~1 week	Grade lifts to A. Regulatory compliance.

5.2 Cross-Feature Dependencies

- ✓ Feature #7 (Login Flow) — the login form is the input layer; #7 corrected 3 implementation defects but the security model is here in #8
- ✓ Feature #9 (Session Management) — the session model depends on #8; httpOnly cookie replaces sessionStorage
- ✓ Feature #40 (Copilot History) — per-user history requires per-user accounts
- ✓ Feature #44 (Recently Viewed) — cross-device sync requires per-user accounts
- ✓ Feature #48 (Favorites) — cross-device sync requires per-user accounts
- ✓ Feature #43 (Notifications) — per-user notification preferences require per-user accounts

5.3 Final Assessment

1 SHARED CODE	0 PER-USER ACCOUNTS	0 MFA / 2FA
D GRADE	~1d PHASE 1 EFFORT	~1w PHASE 2 EFFORT

FINAL VERDICT · D GRADE, 1 DAY FROM C+

Authentication Model is a **D grade feature** — the platform's most critical gap. The #7 Login Flow report corrected three implementation defects (server-side validation DOES exist, error feedback DOES work, lock button DOES work), which improves the implementation grade. But the core security model remains: one shared code (4815162342) for everyone, no per-user identity, no MFA, no audit trail. This is a regulatory blocker for professional clinical use — a platform that cannot attribute actions to specific practitioners cannot be used in a regulated pharmacy setting. Phase 1 (1 day — NextAuth.js migration with httpOnly cookie) closes the «3-second bypass» risk and lifts the grade to C+. Phase 2 (1 week — per-user accounts) unlocks 6 features across the platform (Copilot history, recently-viewed, favorites, notifications, audit trail, cross-device session) and lifts the grade to B. Phase 3 (3 days — TOTP MFA + rate limiting) and Phase 4 (1 week — audit log) lift it to A. This is the single highest-ROI investment in the entire 68-item review menu: 1 week of work, fixes the P0 security hole, unlocks 6 features, and enables regulatory compliance.

Immediate Next Steps

1. Install NextAuth.js (npm install next-auth) — 10 minutes.
2. Create app/api/auth/[...nextauth]/route.ts with Credentials provider — 2 hours.
3. Move the access code to an environment variable (process.env.DZPHARM_ACCESS_CODE) — 10 minutes.
4. Replace sessionStorage.dzpharm_session with NextAuth's useSession hook — 4 hours.
5. Defer per-user accounts (Phase 2) to week 2.
6. Defer MFA + audit log (Phase 3-4) to weeks 3-4.

8

Single shared code, no user identity, no MFA, no audit trail. P0 critical. #7 corrected 3 implementation defects. NextAuth.js migration (1 day) closes bypass. Per-user accounts (1 week) unlock 6 features.

5

CHAPTERS

4

SCREENSHOTS

P0

CRITICAL

D

GRADE
